Lecture 6 - Arrays

Arrays are mutable data structures allocated on the heap whose elements can be accessed via indices. In this lecture, we will examine how to use arrays and how to verify programs that use them. In particular, we will illustrate the method of invariants for arrays and encounter the first specifications involving frames to address aliasing.

Mutable data

In functional programming, values are immutable; this makes the code more perspicuous and facilitates logical reasoning about programs. Unfortunately, when data structures might grow large, it is both space- and time-consuming to generate new values from old ones, even when just local changes would suffice.

By contrast, in ordinary imperative and object-oriented programming languages, allocating persistent data structures on the heap and accessing them via references to modify data in place is the bread and butter of coding.

Variables are the first mutable data structure we have encountered; arrays, formerly called multidimensional variables, e.g., in FORTRAN, are a prominent example of modifiable data structures. Like with C++ and Java, they are allocated in Dafny using the **new** instruction, and are accessed via integer indexes in the range 0..array-name.Length - 1.

Arrays: allocation, reference, comparison with sequences.

```
method TestArray(j: nat, k: nat)
    requires j < 10 && k < 10
{
    var a := new int[10];
    // allocates an array a[0..a.Length - 1]
    // of integers on the heap; then a points to it
    assert a.Length == 10;
    a[j] := 60;
    a[k] := 65;
    if j == k {
        assert a[j] == 65;
    } else {
        assert a[j] == 60;
    }
}
```

As with C, Java, and Python, references can be copied and passed, creating aliases to the same data structure. In the example below, b is an alias of a. Therefore, changes made via either will be visible when accessing the same entries from the other; this is called a *side effect*.

```
method ArraysAreReferences()
{
    var a := new string[20];
    a[7] := "hello";
    var b := a; // aliasing: a and b refer to the same array
    assert b[7] == "hello";
    b[7] := "hi";
    a[8] := "greetings";
    assert a[7] == "hi" && b[8] == "greetings";
}
```

Dafny includes sequence types: seq<T> (where T is any type). While arrays are reference types, sequences are value types and hence unmodifiable. Nonetheless, it will be useful to convert an array to the sequence of its entries when specifying methods and functions.

```
method ArraysVsSequences()
{
    var greetings : seq<string>;
    greetings := ["hey", "hola", "tjena"];
    // sequences are immutable values; the two lines above
    // have the same effect as
    var greetings' := ["hey", "hola", "tjena"];
    assert greetings == greetings';

    // the length of a sequence s is written |s|
    assert |greetings| == 3;
    // elements of a sequence are accessed via indexes, as for arrays
    assert greetings[2] == "tjena";
    // sequences can be concatenated using _+_
    assert [1, 5, 12] + [22, 35] == [1, 5, 12, 22, 35];
```

If a is a sequence or an array and 0 <= lo <= hi <= |a| or a.Length, then a[lo..hi] == [a[lo], ... , a[lo - 1]] is the sequence (of type seq) consisting of hi - lo entries from a[]. If lo is omitted, it defaults to 0; if hi is omitted, it defaults to |a| or a.Length. a[..] is the entire sequence:

```
    var a := new int[3];
    a[0], a[1], a[2] := 6, 28, 496;
    assert a[..2] == [6, 28] && a[1..] == [28, 496];
    var s := a[..];
    assert s == [6, 28, 496];
    a[0] := 8128;
    assert s[0] == 6;

    // s is a sequence, hence no mutation may happen
}
```

Searching arrays

A common task with arrays is searching for elements that satisfy a given property P; note that Dafny allows higher-order types for parameters, including P, whose type T -> bool is functional, with domain T (which is generic) and range bool.

The method LinearSearch1(a, P) returns either an index n of an element a[n] satisfying P or a.Length if it does not exist.

```
method LinearSearch1<T>(a: array<T>, P: T -> bool) returns (n: int)
    ensures 0 <= n <= a.Length
    ensures n == a.Length || P(a[n])
```

Without the next clause, the method returning just a.Length would check! This is a case of under-specification, which we fix by adding the condition under which a.Length is correct:

```
    ensures n == a.Length ==>
        forall i :: 0 <= i < a.Length ==> !P(a[i]) // slightly verbose
{
    n := 0;
    while n != a.Length
        invariant 0 <= n <= a.Length
        invariant forall i :: 0 <= i < n ==> !P(a[i])
    {   if P(a[n]) {
            return;
        }
        n := n + 1;
    }
}
```

The specification for the next version explicitly states that we are returning the leftmost a[n] such that P, if any (we have avoided the nested if):

```
method LinearSearch2<T>(a: array<T>, P: T -> bool) returns (n: int)
    ensures 0 <= n <= a.Length
    ensures n == a.Length || P(a[n])
    ensures forall i :: 0 <= i < n ==> !P(a[i])
{
    n := 0;
    while n != a.Length && !P(a[n])
        invariant 0 <= n <= a.Length
        invariant forall i :: 0 <= i < n ==> !P(a[i])
    {   // all the subsequent assertions are unnecessary to verify
        // that the body preserves the (interesting part of the)
        // invariant
        assert forall i :: 0 <= i < n ==> !P(a[i]) && !P(a[n]);
        assert forall i :: 0 <= i < n + 1 ==> !P(a[i]);
        n := n + 1;
        assert forall i :: 0 <= i < n ==> !P(a[i]);
    }
}
```

The final version of LinearSearch assumes that an n s.t. P(a[n]) exists and illustrates the usage of the existential quantifier:

```
method LinearSearch3<T>(a: array<T>, P: T -> bool) returns (n: int)
    requires exists i :: 0 <= i < a.Length && P(a[i])
    ensures  0 <= n < a.Length && P(a[n])
{
    n := 0;
    while true
        invariant 0 <= n < a.Length
        invariant exists i :: n <= i < a.Length && P(a[i])
        decreases a.Length - n
    {   // again the following assertions are unnecessary
        assert exists i :: n <= i < a.Length && P(a[i]);
        if P(a[n]) { return; }
        assert !P(a[n]);
        assert exists i :: n < i < a.Length && P(a[i]);
        assert exists i :: n + 1 <= i < a.Length && P(a[i]);
        n := n + 1;
        assert exists i :: n <= i < a.Length && P(a[i]);
    }
}
```

Linear search requires time O(n), meaning it is bounded everywhere except for a finite number of cases by a linear function of the array length. If the array is sorted, however, we can speed up the search for an element using the binary search algorithm, which runs in time O(log n).

BinarySearch will return an integer n, which is the "earliest insertion point" in the array a[], namely the index of the leftmost occurrence of the key if it exists, n == a.Length otherwise

```
method BinarySearch(a: array<int>, key: int) returns (n: nat)
    requires forall i, j :: 0 <= i < j < a.Length ==> a[i] <= a[j]
    ensures  0 <= n <= a.Length
    ensures  forall i :: 0 <= i < n ==> a[i] < key
    ensures  forall i :: n <= i < a.Length ==> key <= a[i]
{
    var lo, hi := 0, a.Length;
    while lo < hi
        invariant 0 <= lo <= hi <= a.Length
        invariant forall i :: 0 <= i < lo ==> a[i] < key
        invariant forall i :: hi <= i < a.Length ==> key <= a[i]
    {
        var mid := (lo + hi) / 2;
        if a[mid] < key {
            lo := mid + 1;
            // with lo := mid the loop
            // might not terminate: take lo == hi - 1
        } else {
            hi := mid;
        }
    }
    n := lo;
}
```

Modifying arrays

Because of aliasing, reasoning about arrays when their entries are modified is hard. Dafny uses *dynamic frames* to handle such situations; the expression "modifies a" below is a write frame specifying what the method is allowed to

modify. It declares that the method can modify the entries of the array referred to by a, not the array itself.

```
method SetEndPoints(a: array<int>, left: int, right: int)
    requires a.Length != 0
    modifies a
{
    a[0] := left;
    a[a.Length - 1] := right;
}
```

In the next example, the write frame "modifies a" extends to c because c is an alias of a; it extends to b if b is also an alias of a, which is checked in the if condition; without this, the statement b[10] := b[0] + 1; does not verify.

```
method Aliases(a: array<int>, b: array<int>)
    requires 100 <= a.Length
    modifies a
{
    a[0] := 10;
    var c := a;
    if b == a {
        b[10] := b[0] + 1;
    }
    c[20] := a[14] + 2;
}
```

When specifying a method that is given the right to modify an array, it might be necessary to refer to the entry values before the modification happens, which is called the "old" state or pre-state. In addition to using ghost variables of type seq, namely sequences of values of type T, in the body of the method to save the values of the array entries in the pre-state, Dafny provides a special reference to them via the old operator.

More precisely, an expression E in the ensures clause refers to the method's post-state; to refer to the meaning of E in the pre-state, one must use old(E) instead.

```
method UpdateEelemnts(a: array<int>)
    requires a.Length == 10
    modifies a
    ensures  old(a[4]) < a[4]
    ensures a[6] <= old(a[6])
    ensures a[8] == old(a[8])
{
    a[4], a[8] := a[4] + 3, a[8] + 1;
    a[7], a[8] := 516, a[8] - 1;
}
```

In the next example

```
method Increment(a: array<int>, i: int)
    requires 0 <= i < a.Length
    modifies a
```

The clause **ensures** a[i] == old(a)[i] + 1 would be incorrect, as old(a) refers to the reference a in the pre-state, which is unchanged by the method, not to the old value of a[i]; the correct phrasing is:

```
    ensures a[i] == old(a[i]) + 1
{
    a[i] := a[i] + 1;
}
```

Sorting arrays

As a further example of array modification, we present the selection sort algorithm for an array of integers below. To ensure the sameness condition we met while looking at the insertion sort algorithm on lists, we use the multiset facility of Dafny: see the documentation on dafny.org.

A (finite) multiset is like a set that may contain multiple copies of the same element, rather than at most one. Mathematically, a multiset is a mapping from a domain (in the example below, the integers) to the natural numbers, say M : int -> nat; we interpret M(x) == n as the statement that there are n copies of x in M; consistently, if M(x) == 0, we say that x does not belong to M. In the code below, a[..] is the sequence of all values in the entries of a[], and multiset(a[..]) is the multiset of the values in the sequence a[..]; hence, in the second ensure clause, old(multiset(a[..])) refers to the multiset of values of the entries of a[] in the pre-state, while multiset(a[..]) refers to the multiset of the values of a[] in the post-state. Since these two multisets are compared by the number of copies of the elements they contain, their equality implies that the sequence of values in the modified a[] is a permutation of the sequence of the original ones.

```
method SelectionSort(a: array<int>)
    modifies a
    ensures forall i, j :: 0 <= i < j < a.Length
                           ==> a[i] <= a[j]
    ensures multiset(a[..]) == old(multiset(a[..]))
{
    var n := 0;
    while n != a.Length
        invariant 0 <= n <= a.Length
        invariant forall i, j :: 0 <= i < j < n
                           ==> a[i] <= a[j]
        invariant forall i, j :: 0 <= i < n <= j < a.Length
                           ==> a[i] <= a[j]
        invariant multiset(a[..]) == old(multiset(a[..]))
    {
        var mindex, m := n, n;
        while m != a.Length
            invariant n <= m <= a.Length &&
                      n <= mindex < a.Length
            invariant forall i :: n <= i < m
                      ==> a[mindex] <= a[i]
        {
            if a[m] < a[mindex] {
             mindex := m;
            }
            m := m + 1;
        }
        a[n], a[mindex] := a[mindex], a[n];
        n := n + 1;
    }
}
```

Two-state frames: Quicksort

A more sophisticated example of frame usage is Quicksort. The algorithm is based on partitioning the array into two subarrays such that all values in the leftmost subarray are less than or equal to those in the rightmost subarray. The partition is constructed around an arbitrarily chosen element, called the pivot, which is placed at the end of the leftmost subarray; its position is referred to as a *split point* of the array.

```
// SplitPoint(a, n) iff
//    forall i in 0..n and j in n..a.Length we have a[i] <= a[j]
ghost predicate SplitPoint(a: array<int>, n: int)
    requires 0 <= n <= a.Length
    reads a
{
    forall i, j :: 0 <= i < n <= j < a.Length ==> a[i] <= a[j]
}
```

The sorting algorithm permutes the elements of the array by partitioning them around the pivot, then recursively calling itself on the two parts obtained in this way. To specify and verify the partition method, as well as the whole quicksort program, we have to check that when a portion of the array is permuted, the values outside that portion remain the same and in the same order. This involves defining a predicate that depends on two states, namely the pre-state and the post-state, in the execution of the calling context. In Dafny, such a predicate is declared as two-state, as exemplified in the SwapFrame predicate.

```
// SwapFrame(a, lo, hi) iff the parts a[..lo] and a[hi..] are unchanged
// from the prestate to the poststate, while the elements of a[..] are the same
// in the pre- and post-state, but possibly permuted
twostate predicate SwapFrame(a: array<int>, lo: int, hi: int)
    requires 0 <= lo <= hi <= a.Length
    reads a
{
    (forall i :: (0 <= i < lo || hi <= i < a.Length) ==>
        a[i] == old(a[i])) &&
    multiset(a[..]) == old(multiset(a[..]))
}
```

We are now in a position to code and specify the Quicksort algorithm. To keep its signature and specifying the same as that of the SelectionSort, we write:

```
method Quicksort(a: array<int>)
    modifies a
    ensures forall i, j :: 0 <= i < j < a.Length ==> a[i] <= a[j]
    ensures multiset(a[..]) == old(multiset(a[..]))
{
    QuicksortAux(a, 0, a.Length);
}
```

where QuicksortAux(a, lo, hi) recursively sorts the subarray a[lo..hi]:

```
method QuicksortAux(a: array<int>, lo: int, hi: int)
    requires 0 <= lo <= hi <= a.Length
    requires SplitPoint(a, lo) && SplitPoint(a, hi)
    modifies a
    ensures forall i, j :: lo <= i < j < hi ==> a[i] <= a[j]
    ensures SwapFrame(a, lo, hi)
    ensures SplitPoint(a, lo) && SplitPoint(a, hi)
    decreases hi - lo
{
    if lo < hi {
        var p := Partition(a, lo, hi);
        QuicksortAux(a, lo, p);
        QuicksortAux(a, p + 1, hi);
    }
}
```

The central step is the call to the method Partition(a, lo, hi), which partitions a[lo..hi] around the pivot a[lo] and returns the position p where the pivot has been placed.

```
method Partition(a: array<int>, lo: int, hi: int) returns (p: int)
    requires 0 <= lo < hi <= a.Length
    requires SplitPoint(a, lo) && SplitPoint(a, hi)
    modifies a
    ensures lo <= p < hi
    ensures forall i :: lo <= i < p ==> a[i] <= a[p]
    ensures forall i :: p <= i < hi ==> a[p] <= a[i]
    ensures SplitPoint(a, lo) && SplitPoint(a, hi)
    ensures SwapFrame(a, lo, hi)
{
    var pivot := a[lo];
    var m, n := lo + 1, hi;
    while m < n
        invariant lo + 1 <= m <= n <= hi
        invariant a[lo] == pivot
        invariant forall i :: lo + 1 <= i < m ==> a[i] <= pivot
        invariant forall i :: n <= i < hi ==> pivot < a[i]
        invariant SplitPoint(a, lo) && SplitPoint(a, hi)
        invariant SwapFrame(a, lo, hi)
    {
        if a[m] <= pivot {
            m := m + 1;
        } else if a[n - 1] > pivot {
            n := n - 1;
        } else {
            assert a[m] > pivot && a[n - 1] <= pivot;
            a[m], a[n - 1] := a[n - 1], a[m];
            m, n := m + 1, n - 1;
        }
    }
    a[lo], a[m - 1] := a[m - 1], a[lo];
    p := m - 1;
}
```

References: Leino, chapters 13, 14, and 15.